CYS5550 Week 7: Computer Vision — From Pixels to Deep Learning

- Course: CYS5550 – Intro to Artificial Intelligence
- Week: 7 of 8
- Session: Tuesday, 5:30 PM – 8:30 PM
- Duration: 3 hours

Module Learning Objectives

By the end of this module, students will be able to:

1. Identify core computer vision tasks and the challenges that make visual understanding difficult
2. Explain rule-based and classical approaches to CV with their strengths and limitations
3. Describe feature engineering and classical machine learning methods for image understanding
4. Trace the evolution from hand-crafted features to learned representations
5. Articulate how Convolutional Neural Networks revolutionized computer vision
6. Compare different approaches (rules, classical ML, deep learning) for the same CV task
7. Evaluate when to use different CV approaches based on requirements and constraints

Recurring Example: Robotic Bin Picking

Throughout this lecture, we will trace how a single industrial task—robotic bin picking—has been solved across different AI eras:

```
BIN PICKING ACROSS THREE ERAS
═══════════════════════════════════════════════════════════════════════

  WHAT IS BIN PICKING?
  ─────────────────────────────────────────────────────────────────────
  A robot must identify, locate, and grasp individual objects
  from a bin (container) of randomly arranged parts.

  ┌─────────────────────────────────────────────────────────────────────┐
  │                         BIN PICKING SUBTASKS                        │
  ├──────────────────┬──────────────────┬──────────────────┬──────────────────┤
  │  OBJECT          │  OBJECT          │  POSE            │  GRASP           │
  │  DETECTION       │  RECOGNITION     │  ESTIMATION      │  PLANNING        │
  │                  │                  │                  │                  │
  │  "Where are      │  "Is this a      │  "What is its    │  "How should     │
  │   objects in     │   bolt, nut,     │   3D position    │   the gripper    │
  │   this image?"   │   or washer?"    │   & rotation?"   │   approach?"     │
  │                  │                  │                  │                  │
  └──────────────────┴──────────────────┴──────────────────┴──────────────────┘
                     │
     ┌───────────────┼───────────────┐
     ▼               ▼               ▼

  ┌─────────┐    ┌─────────┐    ┌─────────┐
```

```
    │  ERA 1  │    │  ERA 2  │    │  ERA 3  │
    │Classical│    │ Feature │    │  Deep   │
    │   CV    │    │ Eng.+ML │    │Learning │
    │ 1960-95 │    │ 1995-12 │    │ 2012+   │
    └─────────┘    └─────────┘    └─────────┘
```

WHY BIN PICKING?

```
═══════════════════════════════════════════════════════

 It is the "Hello World" of industrial computer vision:
 ─────────────────────────────────────────────────────
 • Requires detection, recognition, localization, AND planning
 • Has been attempted in every era of CV
 • Exposes the strengths and weaknesses of each approach
 • Real-world economic impact (billions in manufacturing)
 • Still an active research area today!

 DIFFICULTY SPECTRUM:
 ─────────────────────────────────────────────────────
 EASY ◄──────────────────────────────────────► HARD

 Structured parts     Mixed parts       Random pile
 known orientation    some overlap       heavy occlusion
 good lighting        variable light     reflective/dark
 single object type   few types          unknown objects
```

Part 1: Computer Vision Foundations & Classical Approaches

1.1 What is Computer Vision?

Definition and Scope

Computer Vision (CV) is the field of AI concerned with giving computers the ability to interpret and understand visual information from the world— images, video, and 3D data.

```
THE COMPUTER VISION LANDSCAPE
═══════════════════════════════════════════════════════
               COMPUTER VISION
                     │
     ┌───────────────┼───────────────┐
     │               │               │
     ▼               ▼               ▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│  PERCEPTION  │ │ UNDERSTANDING │ │  GENERATION  │
│              │ │              │ │              │
│ • Edge detect │ │ • Classify   │ │ • Image gen  │
```

```
│ • Segment     │ │ • Detect obj  │ │ • Style xfer  │
│ • Depth est.  │ │ • Recognize   │ │ • Super-res   │
│ • Optical flow│ │ • Track       │ │ • Inpainting  │
└───────────────┘ └───────────────┘ └───────────────┘
```

Why is Vision Hard for Computers?

```
THE CHALLENGES OF COMPUTER VISION
═══════════════════════════════════════════════════════

1. VIEWPOINT VARIATION
   ──────────────────────────────────────────────────
    A bolt looks very different from the side vs. top-down
    Same object → completely different pixel patterns

       ┌────┐       ┌──────────┐       •
       │    │       │          │      /|\
       │bolt│       │   bolt   │     / | \
       └────┘       └──────────┘    bolt (end-on)
      (side)      (top-down)

2. ILLUMINATION CHANGES
   ──────────────────────────────────────────────────
    Same part under different lighting:
    • Bright light → washed out, reflections
    • Dim light   → noisy, low contrast
    • Directional → strong shadows change shape appearance

    In a factory, lighting shifts with time of day,
    machine vibration, and part reflectivity.

3. OCCLUSION (Objects blocking each other)
   ──────────────────────────────────────────────────
    In a bin, parts overlap and hide each other!

    ┌──────────────────┐
    │  ┌──┐            │
    │  │  │  ┌──┐      │   Only partial views
    │  └──┘──│  │      │   of each object
    │     ┌──┴──┴──┐   │
    │     │        │   │
    │     └────────┘   │
    └──────────────────┘

4. INTRA-CLASS VARIATION
   ──────────────────────────────────────────────────
    "Bolts" come in many sizes, colors, materials
    The category is the same, but appearance varies widely

5. SCALE VARIATION
   ──────────────────────────────────────────────────
```

```
    An M3 bolt vs. an M16 bolt—both are "bolts"
    but pixel footprint changes dramatically
    with distance and object size

6. BACKGROUND CLUTTER
   ──────────────────────────────────────────────────
    Metallic parts in a metallic bin—
    object and background look the same!

7. DEFORMATION
   ──────────────────────────────────────────────────
    Cables, gaskets, rubber parts change shape
    when piled together → same object, different geometry

CONNECTION TO BIN PICKING:
   ──────────────────────────────────────────────────
    Bin picking encounters ALL of these challenges
    simultaneously, making it one of the hardest
    practical CV problems.
```

Core Computer Vision Tasks

```
TAXONOMY OF COMPUTER VISION TASKS
═══════════════════════════════════════════════════════════

IMAGE-LEVEL (Label the whole image)
──────────────────────────────────────────────────
 • Image Classification: "This is a bolt"
 • Scene Recognition: "This is a factory floor"
 • Quality Inspection: "This part is defective / OK"

OBJECT-LEVEL (Find and label objects)
──────────────────────────────────────────────────
 • Object Detection: Draw bounding box + class label
   [bolt] [nut] [washer] in image
 • Object Recognition: Identify specific instance
   "M6×20 hex bolt" vs "M8×30 hex bolt"
 • Object Tracking: Follow object across video frames

PIXEL-LEVEL (Label every pixel)
──────────────────────────────────────────────────
 • Semantic Segmentation: Every pixel → class
   (bolt pixels, nut pixels, background pixels)
 • Instance Segmentation: Separate individual objects
   bolt_1 pixels vs. bolt_2 pixels
 • Depth Estimation: Each pixel → distance from camera

3D UNDERSTANDING (Recover geometry)
──────────────────────────────────────────────────
 • Pose Estimation: Object position + orientation in 3D
 • 3D Reconstruction: Build 3D model from images
```

```
  • Point Cloud Processing: Analyze 3D sensor data

BIN PICKING REQUIRES:
──────────────────────────────────────────────────

 Detection → "Where are objects?"
 Recognition → "What type is each one?"
 Segmentation → "Exact shape boundaries?"
 Pose Estimation → "How is it oriented in 3D?"
 All feeding into → GRASP PLANNING
```

1.2 Images as Data

How Computers See Images

```
DIGITAL IMAGES: WHAT THE COMPUTER ACTUALLY SEES
═══════════════════════════════════════════════════════

AN IMAGE IS A GRID OF NUMBERS
──────────────────────────────────────────────────

  What you see:           What the computer sees:

    ┌──────────┐           ┌──────────────────────┐
    │  (photo  │           │ 142  138  135  129 ..│
    │   of a   │           │ 145  141  137  132 ..│
    │   bolt)  │           │ 148  144  140  136 ..│
    │          │           │ 150  147  143  139 ..│
    └──────────┘           └──────────────────────┘

   Grayscale: Each pixel = 1 number (0-255)
              0 = black, 255 = white

   Color (RGB): Each pixel = 3 numbers
              Red channel:   [142, 138, 135, ...]
              Green channel: [140, 136, 133, ...]
              Blue channel:  [ 50,  48,  45, ...]

IMAGE DIMENSIONS:
──────────────────────────────────────────────────
 A 640×480 RGB image = 640 × 480 × 3 = 921,600 numbers!
 A 1920×1080 image = 1920 × 1080 × 3 = 6,220,800 numbers!

 Compare to NLP:
   Sentence "The cat sat" = 3 tokens (words)
   Small image = ~1 million values
   → Vision data is MUCH higher-dimensional!
```

DEPTH IMAGES (used in bin picking):

```
──────────────────────────────────────────────────
 RGB camera → color at each pixel
 Depth camera → distance at each pixel (in mm)

 ┌──────────────────────┐
 │ 450  448  445  442 ..│   Each number = distance
 │ 452  451  447  443 ..│   from camera in mm
 │ 510  508  505  500 ..│
 │ 512  511  508  504 ..│   Closer objects = smaller values
 └──────────────────────┘   Jump in values = object edge!

 RGB-D cameras (e.g., Intel RealSense, Microsoft Kinect)
 give BOTH color AND depth → essential for bin picking
```

1.3 Era 1: Classical Computer Vision (1960s–1990s)

The Philosophy

Core Belief: Vision can be solved by detecting geometric primitives (edges, corners, regions) and reasoning about their arrangement. If we can find edges and shapes, we can recognize objects.

```
CLASSICAL CV ARCHITECTURE
═══════════════════════════════════════════════════════

   Input Image
       │
       ▼
   ┌─────────────────┐
   │  PREPROCESSING  │  ← Noise removal, contrast
   └─────────────────┘
           │
           ▼
   ┌─────────────────┐
   │ EDGE DETECTION  │  ← Find boundaries (Canny, Sobel)
   └─────────────────┘
           │
           ▼
   ┌─────────────────┐
   │    SHAPE        │  ← Lines, circles, contours
   │    DETECTION    │     (Hough transform)
   └─────────────────┘
           │
           ▼
   ┌─────────────────┐
   │   TEMPLATE      │  ← Match against known patterns
   │   MATCHING      │
   └─────────────────┘
           │
           ▼
   ┌─────────────────┐
```

```
    │   GEOMETRIC     │  ← Reason about spatial
    │   REASONING     │     relationships
    └─────────────────┘
            │
            │
            ▼
      Object Identity & Pose
```

Edge Detection: The Foundation

```
EDGE DETECTION
═══════════════════════════════════════════════════

IDEA: Edges = rapid changes in pixel intensity
      Edges correspond to object boundaries!

SOBEL OPERATOR (gradient-based):
──────────────────────────────────────────────

  Horizontal gradient kernel:     Vertical gradient kernel:
  ┌────┬────┬────┐                ┌────┬────┬────┐
  │ -1 │  0 │ +1 │                │ -1 │ -2 │ -1 │
  ├────┼────┼────┤                ├────┼────┼────┤
  │ -2 │  0 │ +2 │                │  0 │  0 │  0 │
  ├────┼────┼────┤                ├────┼────┼────┤
  │ -1 │  0 │ +1 │                │ +1 │ +2 │ +1 │
  └────┴────┴────┘                └────┴────┴────┘

  Slide kernel across image:
    At each pixel, multiply neighbors by kernel weights and sum.
    Large value = strong edge!
```

```
CANNY EDGE DETECTOR (1986):
──────────────────────────────────────────────

  1. Smooth image (Gaussian blur to reduce noise)
  2. Compute gradients (Sobel in x and y)
  3. Non-maximum suppression (thin edges to 1 pixel)
  4. Hysteresis thresholding (keep strong edges,
     follow weak edges connected to strong ones)

  Result: Clean, 1-pixel-wide edges

  INPUT IMAGE         →      CANNY OUTPUT
  ┌──────────────┐           ┌──────────────┐
  │   (photo     │           │              │
  │    of a      │    →      │    ╭──╮      │
  │    bolt)     │           │    │  │      │
  │              │           │    ╰──╯─╮    │
  │              │           │         ╰─╮  │
  └──────────────┘           └──────────────┘
```

Template Matching

```
TEMPLATE MATCHING
═══════════════════════════════════════════════════════

IDEA: Slide a small reference image (template) across the
      scene image. Find where it matches best.

ALGORITHM:
──────────────────────────────────────────────────
  For every position (x, y) in the scene:
    1. Extract patch of template size
    2. Compare patch to template (correlation or SSD)
    3. Record similarity score

  score(x,y) = Σ |scene_patch(i,j) - template(i,j)|²

  Minimum score = best match location!

  Scene Image:                  Template:
  ┌────────────────────┐       ┌──────┐
  │                    │       │(bolt│
  │        ╳ ← match!  │       │ img)│
  │                    │       └──────┘
  │                    │
  └────────────────────┘
```

BIN PICKING APPLICATION:

1. Capture image of bin from fixed camera above
2. Use template of known part (bolt, nut, etc.)
3. Slide template across image
4. Peaks in correlation map = part locations

LIMITATIONS:

- ✗ Only works if part appears at SAME scale
- ✗ Only works if part appears at SAME rotation
- ✗ Sensitive to lighting changes
- ✗ Cannot handle occlusion (partial views)
- ✗ Slow for large images / many templates
- Need separate template for EACH:
    - Scale (small, medium, large)

- Rotation (0°, 10°, 20°, ... 350°)
- Object type
- = combinatorial explosion!

Hough Transform: Finding Shapes

```
HOUGH TRANSFORM
═══════════════════════════════════════════════════════

IDEA: Detect geometric shapes (lines, circles) by voting
      in parameter space

HOUGH CIRCLE DETECTION (for round parts):
──────────────────────────────────────────────────

  Washers and bolt heads are circles!

  For each edge pixel (x, y):
    "Vote" for all circles that could pass through it

  Parameters: center_x, center_y, radius
  Accumulator array counts votes

  Edge pixels from a circle all vote for the same
  (center_x, center_y, radius) → peak in accumulator!

  ┌──────────────────┐      ┌──────────────────────┐
  │                  │      │    Accumulator:      │
  │     ○   ○        │  →   │    Peak at (120,80)  │
  │       ○          │      │    radius = 25       │
  │                  │      │   = washer detected! │
  └──────────────────┘      └──────────────────────┘
```

BIN PICKING: CLASSICAL APPROACH

```
═══════════════════════════════════════════════════════

  Camera Image
       │
       ▼
  ┌──────────────┐
  │ Preprocessing│  Blur, threshold, contrast
  └──────────────┘
       │
       ▼
  ┌──────────────┐
  │Edge Detection│  Canny edges
  └──────────────┘
       │
```

```
          │
          ▼
    ┌─────────────┐
    │ Hough / Blob │  Find circles (washers), lines (bolts)
    │  Detection   │
    └─────────────┘
          │
          ▼
    ┌─────────────┐
    │  Template    │  Match against known part templates
    │  Matching    │  at multiple rotations / scales
    └─────────────┘
          │
          ▼
    ┌─────────────┐
    │  Geometric   │  Compute 2D position
    │  Reasoning   │  Estimate orientation from edges
    └─────────────┘
          │
          ▼
   Robot picks part (approximate 2D pose only)
```

RESULTS (1970s–1990s industrial systems):

- Worked for STRUCTURED bins (parts in known layers)
- Required CONTROLLED lighting (dedicated enclosure)
- Limited to SIMPLE shapes (circles, rectangles)
- Needed FIXED camera position
- Could NOT handle random piles → "random bin picking" remained unsolved!

Famous systems: Adept, FANUC early vision-guided systems

Strengths and Limitations of Classical CV

STRENGTHS:

- ✓ Explainable: Know exactly why a detection occurred
- ✓ No training data needed: Works from mathematical definitions
- ✓ Real-time: Simple operations are very fast
- ✓ Precise: Sub-pixel accuracy for well-defined features
- ✓ Predictable: Same input always gives same output
- ✓ Low compute: Runs on embedded hardware

LIMITATIONS:

- ✗ Brittle to lighting changes
- ✗ Cannot handle complex textures or cluttered scenes
- ✗ No semantic understanding ("this is a bolt")
- ✗ Manual parameter tuning per application
- ✗ Fails on occlusion

- ✗ Poor generalization to new objects

THE FUNDAMENTAL PROBLEM:

- Pixels are not objects!
  - Same object → different pixels (viewpoint, lighting)
  - Different objects → similar pixels (color, texture)
  - No amount of edge detection bridges this gap
- Classical CV hit a ceiling on complex recognition tasks. This motivated the shift to FEATURE-BASED methods...

Part 2: Feature Engineering & Machine Learning for Vision

2.1 Era 2: Feature Engineering + ML (1995–2012)

The Feature Engineering Paradigm

```
FEATURE-BASED CV: THE PIPELINE
═══════════════════════════════════════════════════════

    Raw Image
       │
       ▼
  ┌─────────────────┐
  │  FEATURE        │  Detect interest points or
  │  DETECTION      │  regions (THE HARD PART!)
  └─────────────────┘
          │
          ▼
  ┌─────────────────┐
  │  FEATURE        │  Compute descriptor vector
  │  DESCRIPTION    │  for each point/region
  └─────────────────┘
          │
          ▼
  ┌─────────────────┐
  │  ML ALGORITHM   │  SVM, Random Forest,
  │                 │  k-NN, Boosting...
  └─────────────────┘
          │
          ▼
      Prediction

KEY INSIGHT: The choice of FEATURES determines success!
   Just like in NLP—representation matters more than algorithm.
```

Local Feature Descriptors

```
SIFT: SCALE-INVARIANT FEATURE TRANSFORM
═══════════════════════════════════════════════════════════

(Lowe, 1999/2004)

IDEA: Find distinctive "keypoints" in images that are
      stable across scale, rotation, and lighting changes.

HOW SIFT WORKS:
──────────────────────────────────────────────────

  1. SCALE-SPACE DETECTION
     Build image pyramid (blur + downsample)
     Find extrema across scales → keypoint locations

     Original    Blurred    More Blurred
     ┌─────┐    ┌─────┐    ┌─────┐
     │ . . │    │  .  │    │     │
     │  ★  │    │  ★  │    │  ★  │    ★ = stable
     │ . . │    │  .  │    │     │       keypoint
     └─────┘    └─────┘    └─────┘

  2. KEYPOINT LOCALIZATION
     Refine position to sub-pixel accuracy
     Remove low-contrast and edge responses

  3. ORIENTATION ASSIGNMENT
     Compute dominant gradient direction at keypoint
     → Rotation invariance!

  4. DESCRIPTOR COMPUTATION
     In 16×16 patch around keypoint:
     Divide into 4×4 grid of cells
     In each cell, compute 8-bin gradient histogram
     → 4×4×8 = 128-dimensional descriptor vector

     ┌────┬────┬────┬────┐
     │ ↑→ │ →↓ │ ↓← │ ←↑ │   Each cell: 8 gradient
     ├────┼────┼────┼────┤   directions histogram
     │ →↓ │ ↓← │ ←↑ │ ↑→ │
     ├────┼────┼────┼────┤
     │ ↓← │ ←↑ │ ↑→ │ →↓ │   Total: 4×4×8 = 128
     ├────┼────┼────┼────┤   floating-point values
     │ ←↑ │ ↑→ │ →↓ │ ↓← │
     └────┴────┴────┴────┘

RESULT: Each keypoint has:
   • (x, y) location
   • scale
   • orientation
   • 128-dim descriptor that is invariant to
     scale, rotation, and partially to lighting
```

WHY SIFT MATTERS FOR BIN PICKING:

- Find SAME bolt at different rotations ✓
- Match despite scale changes ✓
- Partial occlusion: enough keypoints survive ✓
- Match new image against database of known parts
- Procedure:
    1. Build keypoint database for each known part
    2. Extract SIFT from bin image
    3. Match keypoints → identify which part is where

Other Important Feature Descriptors

```
FEATURE DESCRIPTOR ZOO
═══════════════════════════════════════════════════════

SURF (Speeded-Up Robust Features, 2006):
──────────────────────────────────────────────────
  • Faster approximation of SIFT
  • Uses box filters and integral images
  • 64-dim descriptor (vs. SIFT's 128)
  • ~3× faster than SIFT, similar performance

ORB (Oriented FAST and Rotated BRIEF, 2011):
──────────────────────────────────────────────────
  • Binary descriptor (256 bits instead of 128 floats)
  • VERY fast to compute and compare
  • Free to use (SIFT/SURF had patents until 2020)
  • Ideal for real-time applications

HOG (Histogram of Oriented Gradients, 2005):
──────────────────────────────────────────────────
  • Designed for object detection (not keypoints)
  • Divide image into cells → gradient histogram per cell
  • Captures shape information

  HOG FOR OBJECT DETECTION:
  ┌──────────────────────────────────────┐
  │  Image    →  HOG features  →   SVM  │
  │  window      (gradient              │ → Object?
  │              histograms)            │   Yes/No
  └──────────────────────────────────────┘

  Slide detection window across image at multiple scales
  HOG + SVM = state-of-the-art pedestrian detection (2005-2012)
```

Feature Matching for Object Recognition

```
FEATURE MATCHING PIPELINE
═══════════════════════════════════════════════════════

  Reference Image          Scene Image
  (known bolt)             (bin with parts)
       │                        │
       ▼                        ▼
  ┌──────────┐            ┌──────────┐
  │ Extract  │            │ Extract  │
  │ SIFT     │            │ SIFT     │
  │ features │            │ features │
  └──────────┘            └──────────┘
       │                        │
       │                        │
       │     ┌──────────┐       │
       └────►│  MATCH   │◄──────┘
             │ features │
             └──────────┘
                  │
                  ▼
       ┌──────────────────┐
       │ GEOMETRIC VERIFY │  RANSAC: find consistent
       │ (RANSAC)         │  geometric transformation
       └──────────────────┘
                  │
                  ▼
     Object location + pose estimate

  RANSAC (Random Sample Consensus):
  ──────────────────────────────────────────────────

    Problem: Some matches are wrong (outliers)
    Solution:
      1. Randomly pick minimal set of matches
      2. Estimate transformation (rotation, translation)
      3. Count how many other matches agree (inliers)
      4. Repeat → keep best transformation

    Robustly estimates pose despite wrong matches!
```

Bag of Visual Words: Image Classification

```
BAG OF VISUAL WORDS (BoVW)
═══════════════════════════════════════════════════════
(Csurka et al., 2004)

IDEA: Borrow "Bag of Words" from NLP for images!

  NLP Bag of Words:        Vision Bag of Visual Words:
  ─────────────────        ───────────────────────────

  Document → word counts   Image → visual word counts
  Vocabulary of words      "Vocabulary" of visual patterns
```

```
  Ignore word order        Ignore spatial layout

HOW IT WORKS:
──────────────────────────────────────────────────

  STEP 1: Build Visual Vocabulary
    • Extract SIFT from thousands of training images
    • Cluster descriptors (k-means, e.g., k=1000)
    • Each cluster center = one "visual word"

  STEP 2: Represent Each Image
    • Extract SIFT from image
    • Assign each descriptor to nearest visual word
    • Count occurrences → histogram

    Image → [0, 3, 0, 7, 1, 0, 5, ...]
             visual word frequencies

  STEP 3: Classify with SVM
    • Train SVM on histograms
    • bolt_histogram vs. nut_histogram vs. washer_histogram
```

```
BIN PICKING APPLICATION:
──────────────────────────────────────────────────

  Train classifier on cropped images of each part type
  At test time:
    1. Detect candidate regions (edge-based or sliding window)
    2. Extract BoVW histogram for each region
    3. SVM classifies: bolt / nut / washer / background

  Accuracy: ~80-90% on structured industrial datasets
```

```
ANALOGY TO NLP:
──────────────────────────────────────────────────

  NLP                      Vision
  ──────────               ──────────
  word                     SIFT descriptor
  vocabulary               visual vocabulary (codebook)
  document = bag of words  image = bag of visual words
  TF-IDF weighting         spatial pyramid matching
  SVM classifier           SVM classifier
```

Sliding Window Detection

```
SLIDING WINDOW OBJECT DETECTION
═══════════════════════════════════════════════════

IDEA: Check every possible location and scale for objects
```

```
ALGORITHM:
──────────────────────────────────────────────────
  For each scale of image pyramid:
    For each position (x, y):
      1. Extract image patch (window)
      2. Compute features (HOG, BoVW, etc.)
      3. Run classifier: Object? Yes/No + confidence
      4. If yes, record bounding box

  ┌──────────────────────────────────────┐
  │ ┌───┐                                │
  │ │? │→ ┌───┐                          │
  │ └───┘  │? │→ ┌───┐                  │
  │        └───┘  │? │→  ... scan entire │
  │               └───┘       image      │
  └──────────────────────────────────────┘

  Then: Non-Maximum Suppression (NMS)
    Remove duplicate overlapping detections
    Keep only the best-scoring box per object

DEFORMABLE PARTS MODEL (DPM, Felzenszwalb 2008-2010):
──────────────────────────────────────────────────
  Model object as root filter + deformable parts

  ┌────────────────────┐
  │   ROOT             │   Coarse object shape
  │  ┌──────┐  ┌────┐  │
  │  │part1│  │pt2│  │   Parts can shift position
  │  └──────┘  └────┘  │   (like bolt head + shaft)
  │      ┌──────┐      │
  │      │part3│      │   Score = root + parts - deformation cost
  │      └──────┘      │
  └────────────────────┘

  This was STATE OF THE ART on PASCAL VOC (~33% mAP) until 2012!
```

Bin Picking with Feature Engineering

```
BIN PICKING: ERA 2 APPROACH
═══════════════════════════════════════════════════════

  RGB-D Camera Image
       │
       ▼
  ┌────────────────────┐
  │ Feature Extract    │  SIFT/SURF keypoints +
  │ (RGB + Depth)      │  3D point features
  └────────────────────┘
           │
           │
```

```
            ▼
    ┌──────────────────┐
    │ Feature Matching │  Compare against CAD model
    │ or Classification│  keypoints or train classifier
    └──────────────────┘
            │
            ▼
    ┌──────────────────┐
    │  Pose Estimation │  RANSAC + ICP (Iterative
    │  (3D)            │  Closest Point)
    └──────────────────┘
            │
            ▼
    ┌──────────────────┐
    │  Grasp Planning  │  Use 3D pose to plan
    │                  │  gripper approach angle
    └──────────────────┘
            │
            ▼
      Robot picks part

  ICP (Iterative Closest Point):
  ──────────────────────────────────────────────────
    Align 3D point cloud from sensor with 3D CAD model
    Iteratively:
      1. Find closest points between clouds
      2. Compute best rigid transform (rotation + translation)
      3. Apply transform, repeat until convergence

    → Precise 6-DOF pose (x, y, z, roll, pitch, yaw)

  RESULTS (2005-2012):
  ──────────────────────────────────────────────────
  • Could handle SOME random bin scenarios
  • Required good 3D sensor data
  • Sensitive to occlusion and reflections
  • Cycle time: 5-15 seconds per pick
  • Success rate: 85-95% on favorable parts
  • Still required extensive per-part tuning
```

Strengths and Limitations of Feature-Based CV

```
FEATURE-BASED CV: ASSESSMENT
═══════════════════════════════════════════════════════

STRENGTHS:
──────────────────────────────────────────────────
 ✓ Invariant to scale, rotation (SIFT/SURF)
 ✓ Handles partial occlusion (enough keypoints survive)
 ✓ Interpretable features (can visualize what was matched)
 ✓ Works with moderate training data
```

```
 ✓ Mathematically grounded

LIMITATIONS:
──────────────────────────────────────────────────

 ✗ Hand-designed features may miss important patterns
 ✗ Features not optimized for the specific task
 ✗ Texture-less objects have few keypoints (plain metal parts!)
 ✗ Sliding window is slow
 ✗ Feature engineering requires domain expertise
 ✗ Performance plateaued around 2010

THE FUNDAMENTAL PROBLEM:
──────────────────────────────────────────────────

  Hand-crafted features cannot capture the full
  complexity of visual appearance.

  What if the FEATURES could be LEARNED from data?
  → This is exactly what deep learning provides.
```

Part 3: Deep Learning for Computer Vision

3.1 The Convolutional Neural Network (CNN)

The Key Idea: Learned Filters

```
CONVOLUTIONAL NEURAL NETWORKS
═══════════════════════════════════════════════════════

RECALL: In classical CV, we designed filters by hand
  (Sobel for edges, Gabor for textures, etc.)

KEY IDEA: Let the network LEARN its own filters from data!

CONVOLUTION OPERATION:
──────────────────────────────────────────────────

  Input Image (5×5):       Learned Filter (3×3):
  ┌───┬───┬───┬───┬───┐    ┌────┬────┬────┐
  │ 1 │ 0 │ 1 │ 0 │ 1 │    │ 1  │ 0  │ -1 │
  ├───┼───┼───┼───┼───┤    ├────┼────┼────┤
  │ 0 │ 1 │ 0 │ 1 │ 0 │    │ 1  │ 0  │ -1 │
  ├───┼───┼───┼───┼───┤    ├────┼────┼────┤
  │ 1 │ 0 │ 1 │ 0 │ 1 │    │ 1  │ 0  │ -1 │
  ├───┼───┼───┼───┼───┤    └────┴────┴────┘
  │ 0 │ 1 │ 0 │ 1 │ 0 │
  ├───┼───┼───┼───┼───┤   Slide filter across image,
  │ 1 │ 0 │ 1 │ 0 │ 1 │   multiply-and-sum at each position
  └───┴───┴───┴───┴───┘   → Output "feature map"

  Same operation as Sobel/Canny, but FILTER VALUES ARE LEARNED!
```

```
CNN ARCHITECTURE:
──────────────────────────────────────────────────

  Input      Conv    Pool   Conv    Pool   Fully-   Output
  Image      Layer   Layer  Layer   Layer  Connected
  ┌────┐   ┌────┐  ┌───┐  ┌────┐  ┌───┐  ┌─────┐  ┌─────┐
  │    │──►│    │─►│   │─►│    │─►│   │─►│     │─►│bolt │
  │RGB │   │feat│  │   │  │feat│  │   │  │class│  │nut  │
  │    │   │maps│  │   │  │maps│  │   │  │     │  │wash.│
  └────┘   └────┘  └───┘  └────┘  └───┘  └─────┘  └─────┘

            edges   reduce  shapes  reduce  combine
            colors  size    parts   size    all info

HIERARCHICAL FEATURE LEARNING:
──────────────────────────────────────────────────

  Layer 1: Learns EDGES and COLORS
           (similar to Sobel, Gabor)

  Layer 2: Learns TEXTURES and SIMPLE SHAPES
           (corners, circles from edges)

  Layer 3: Learns OBJECT PARTS
           (bolt heads, threads, hex patterns)

  Layer 4+: Learns WHOLE OBJECTS
            (complete bolt, nut, washer)

  Each layer builds on the previous one!
  The network automatically discovers the right features.
```

AlexNet: The Deep Learning Moment (2012)

```
ALEXNET AND THE IMAGENET REVOLUTION
═══════════════════════════════════════════════════════

(Krizhevsky, Sutskever, Hinton, 2012)

THE EVENT THAT CHANGED EVERYTHING:
──────────────────────────────────────────────────

  ImageNet Large Scale Visual Recognition Challenge (ILSVRC)
  Task: Classify images into 1,000 categories
  Dataset: 1.2 million training images

  Results in 2012:
  ──────────────────────────────────────────────

  Traditional Methods (HOG+SVM, BoVW):  ~26% error
  AlexNet (CNN):                        ~16% error

  40% RELATIVE IMPROVEMENT!

  Previous years: incremental 1-2% improvements
  2012: CNN smashed the competition by ~10 percentage points
```

```
ALEXNET ARCHITECTURE:
────────────────────────────────────────────────────
  • 8 layers (5 conv + 3 fully connected)
  • 60 million parameters
  • Trained on 2 GPUs for ~6 days
  • Used ReLU activation (key innovation)
  • Dropout for regularization

  Simple by today's standards, but revolutionary in 2012!

IMPACT:
────────────────────────────────────────────────────
  Before 2012: "Deep learning is interesting but impractical"
  After 2012:  EVERY top entry uses CNNs

  Year   Winner         Error    Method
  ──────────────────────────────────────────
  2011   XRCE           25.8%    Fisher Vectors + SVM
  2012   AlexNet        16.4%    CNN (8 layers)
  2013   ZFNet          14.8%    CNN (8 layers, tuned)
  2014   VGGNet/GoogLeNet 7.3%  CNN (19-22 layers)
  2015   ResNet          3.6%    CNN (152 layers!)
  2017   SENet           2.3%    CNN (attention)
  ──────────────────────────────────────────
  Human performance ≈ 5% error

  By 2015, CNNs SURPASSED human accuracy on ImageNet!
```

Key CNN Architectures

```
CNN ARCHITECTURE EVOLUTION
═══════════════════════════════════════════════════════

VGGNet (2014, Simonyan & Zisserman):
────────────────────────────────────────────────────
  • Insight: Use very small (3×3) filters, stack many layers
  • 16-19 layers, 138M parameters
  • Simple, uniform architecture
  • Showed: DEEPER = BETTER (up to a point)

GoogLeNet / Inception (2014, Szegedy et al.):
────────────────────────────────────────────────────
  • "Inception module": Multiple filter sizes in parallel
  • 22 layers, only 5M parameters (efficient!)
  • 1×1 convolutions to reduce computation
  • Showed: ARCHITECTURE DESIGN matters

ResNet (2015, He et al.):
────────────────────────────────────────────────────
  PROBLEM: Very deep networks are hard to train
```

```
            (gradients vanish → learning stalls)

  SOLUTION: "Skip connections" (residual learning)

    Input ──────┐
      │         │
      ▼         │
   [Conv]       │     Instead of learning F(x),
      │         │     learn F(x) + x
      ▼         │
   [Conv]       │     If F is useless, network can
      │         │     learn F=0, output = x (identity)
      ▼         │
    (+) ◄───────┘     ALWAYS at least as good as
      │               shallower network!
      ▼
    Output

  Result: Trained 152-layer network!
          3.6% error on ImageNet (superhuman)

  ResNet is the foundation of most modern CV architectures.
```

Transfer Learning: The Pre-training Revolution

```
TRANSFER LEARNING IN COMPUTER VISION
═══════════════════════════════════════════════════════

KEY INSIGHT (same as BERT in NLP!):
──────────────────────────────────────────────────

  Features learned on ImageNet (1.2M images, 1000 classes)
  are GENERAL-PURPOSE visual features!

  Layer 1 features: Edges → useful for EVERYTHING
  Layer 2 features: Textures → useful for most tasks
  Layer 3+ features: Parts → somewhat task-specific

PRE-TRAIN + FINE-TUNE:
──────────────────────────────────────────────────

  Step 1: Train CNN on ImageNet (expensive, done once)

  Step 2: Adapt to YOUR task:

    ImageNet-trained ResNet (general vision)
           │
           ▼
    ┌──────────────┐
    │ Remove last  │  Remove ImageNet classification head
    │ layer        │
    └──────────────┘
           │
           │
```

```
            ▼
      ┌──────────────┐
      │ Add new head │  Add: bolt / nut / washer classifier
      │ for your task│  (or: defect / no-defect)
      └──────────────┘
            │
            ▼
      ┌──────────────┐
      │ Fine-tune on │  Train on YOUR data (even just
      │ small dataset│  100-1000 images!)
      └──────────────┘

  Result: State-of-the-art with SMALL datasets!

BIN PICKING IMPACT:
──────────────────────────────────────────────────
  Before transfer learning:
    Need 10,000+ labeled images per part type
    Weeks of data collection + annotation

  After transfer learning:
    50-200 images per part type is often sufficient
    Days instead of weeks
    Much easier to deploy new parts!
```

3.2 Object Detection with Deep Learning

From Classification to Detection

```
OBJECT DETECTION: WHERE + WHAT
═══════════════════════════════════════════════════════

Classification:  "This image contains a bolt"
Detection:       "There is a bolt at (x=120, y=80, w=50, h=30)"
                 + class label + confidence score

  Input Image:                  Detection Output:
  ┌──────────────────┐         ┌──────────────────┐
  │                  │         │ ┌──────┐         │
  │  (objects in bin)  │  →    │ │ bolt │   ┌────┐ │
  │                  │         │ └──────┘   │nut │ │
  │                  │         │       ┌────┴────┘ │
  └──────────────────┘         │       │washer│    │
                               │       └──────┘    │
                               └──────────────────┘
```

```
EVOLUTION OF DEEP OBJECT DETECTORS:
──────────────────────────────────────────────────
```

```
R-CNN Family (Two-Stage Detectors):
────────────────────────────────────────────────────

  R-CNN (2014, Girshick):
    1. Propose ~2000 candidate regions (selective search)
    2. For each region: CNN features → SVM classifier
    Slow: ~47 seconds per image!

  Fast R-CNN (2015):
    1. Run CNN once on whole image (shared computation)
    2. Extract features for each region from feature map
    Much faster: ~2 seconds per image

  Faster R-CNN (2015, Ren et al.):
    1. CNN extracts features
    2. Region Proposal Network (RPN) proposes boxes
    3. Classify and refine each proposal
    Real-time capable: ~0.2 seconds per image

    ┌──────────────────────────────────────────┐
    │  Image → CNN →  Feature Map              │
    │           ↓            ↓                 │
    │       RPN (where?)    ROI Pool           │
    │           ↓            ↓                 │
    │      Proposals → Classify + Refine       │
                       ↓                       │
    │             Detections                   │
    └──────────────────────────────────────────┘
```

```
YOLO Family (Single-Stage Detectors):
────────────────────────────────────────────────────

  "You Only Look Once" (Redmon et al., 2016)

  Key idea: ONE pass through the network does everything

    1. Divide image into S×S grid
    2. Each grid cell predicts:
       • B bounding boxes + confidence
       • C class probabilities
    3. Single forward pass → all detections!

   ┌───┬───┬───┐
   │   │ ★ │   │    ★ = grid cell responsible for object
   ├───┼───┼───┤        whose center falls in it
   │   │   │   │
   ├───┼───┼───┤    Predicts: box coordinates, objectness,
   │   │   │   │              class probabilities
   └───┴───┴───┘

  Speed: 45+ FPS (real-time!)
```

```
  Evolution: YOLOv1 → v2 → v3 → v4 → v5 → v7 → v8 → v9 → v11
  Each version: faster AND more accurate
```

```
COMPARISON:
──────────────────────────────────────────────────

  Two-Stage (Faster R-CNN):    Single-Stage (YOLO):
  + More accurate              + Much faster
  + Better for small objects   + Simpler architecture
  - Slower                     - Slightly less accurate
  - More complex               + Real-time capable
```

Instance Segmentation

```
INSTANCE SEGMENTATION: PIXEL-PERFECT DETECTION
══════════════════════════════════════════════════

  Detection:  Bounding box per object
  Instance Segmentation:  Pixel mask per object

  ┌──────────────────────┐     ┌──────────────────────┐
  │  ┌──────┐            │     │ ▓▓▓▓                 │
  │  │ bolt │    ┌────┐  │     │ ▓▓▓▓       ▒▒▒▒      │
  │  └──────┘    │nut │  │     │            ▒▒▒▒      │
  │              └────┘  │     │        ░░▒▒▒▒        │
  └──────────────────────┘     └──────────────────────┘
   Object Detection            Instance Segmentation
   (boxes)                     (pixel masks)
```

```
Mask R-CNN (He et al., 2017):
──────────────────────────────────────────────────

  Faster R-CNN + mask prediction branch

  For each detected object:
    • Bounding box (where)
    • Class label (what)
    • Pixel mask (exact shape)

  Critical for bin picking:
    Exact object boundaries → better grasp planning!
```

3.3 Bin Picking with Deep Learning

Modern Bin Picking Pipeline

```
BIN PICKING: ERA 3 (DEEP LEARNING)
═══════════════════════════════════════════════════════

  RGB-D Camera
       │
       ▼
  ┌──────────────────┐
  │ Deep Object      │  YOLO / Mask R-CNN
  │ Detection +      │  detects & segments
  │ Segmentation     │  each part in the bin
  └──────────────────┘
          │
          │
          ▼
  ┌──────────────────┐
  │ 6-DOF Pose       │  Deep pose estimation
  │ Estimation       │  (e.g., PVNet, DenseFusion)
  │ (Deep Learning)  │  predicts position + orientation
  └──────────────────┘
          │
          │
          ▼
  ┌──────────────────┐
  │ Grasp Prediction │  CNN predicts best grasp
  │ (Deep Learning)  │  point + approach angle
  │                  │  (e.g., GraspNet, Contact-GraspNet)
  └──────────────────┘
          │
          │
          ▼
     Robot executes pick
```

```
VERSUS CLASSICAL APPROACH:
──────────────────────────────────────────────────

  Classical:                      Deep Learning:
  Hand-tuned per part             Learn from data
  Simple shapes only              ANY shape
  Controlled lighting             Robust to lighting
  5-15 sec / pick                 0.5-3 sec / pick
  85-95% success                  95-99%+ success
  Weeks to deploy new part        Hours with fine-tuning
```

```
KEY DEEP LEARNING INNOVATIONS FOR BIN PICKING:
──────────────────────────────────────────────────

  1. SYNTHETIC DATA TRAINING
     • Render thousands of training images from 3D CAD models
     • Randomize: lighting, camera angle, part positions
     • "Domain randomization" bridges sim-to-real gap
     • No manual labeling needed!

  2. DEPTH FUSION
```

```
      • Combine RGB + Depth in early/mid network layers
      • "DenseFusion": fuse pixel-wise RGB + point cloud features
      • Better 3D pose estimation than either modality alone

  3. END-TO-END GRASP LEARNING
      • Input: camera image → Output: grasp command
      • Skip explicit pose estimation entirely!
      • Learn what works from trial and error
```

3.4 Modern Frontiers

Vision Transformers

```
VISION TRANSFORMERS (ViT)
═══════════════════════════════════════════════════════

(Dosovitskiy et al., Google, 2020)

KEY IDEA: Apply Transformer architecture (from NLP!) to images

HOW:
──────────────────────────────────────────────────

  1. Split image into patches (e.g., 16×16 pixels each)
  2. Flatten each patch into a vector
  3. Add position embeddings
  4. Feed through Transformer encoder (self-attention!)

  ┌──┬──┬──┬──┐
  │P1│P2│P3│P4│   Split image into patches
  ├──┼──┼──┼──┤
  │P5│P6│P7│P8│   Each patch = one "token"
  ├──┼──┼──┼──┤   (like a word in NLP!)
  │P9│..│  │  │
  └──┴──┴──┴──┘
       │
       ▼
  [P1] [P2] [P3] ... [CLS]  →  Transformer Encoder
                                       │
                                       ▼
                                  Classification

  Self-attention lets EVERY patch attend to EVERY other patch
  → Global context from the very first layer!
```

```
CNN vs ViT:
──────────────────────────────────────────────────

  CNN: Local → Global (small filters, many layers)
  ViT: Global from start (self-attention over all patches)
```

```
  ViT needs MORE data to train (no inductive bias for locality)
  But with enough data (300M+ images): ViT matches or beats CNNs!
```

Foundation Models for Vision

```
FOUNDATION MODELS: ONE MODEL FOR ALL VISION TASKS
═══════════════════════════════════════════════════

SEGMENT ANYTHING MODEL (SAM, Meta 2023):
────────────────────────────────────────
  • Trained on 11 million images, 1 billion masks
  • Can segment ANY object with a click or box prompt
  • Zero-shot: Works on objects it has never seen!

  Prompt: Click on object → SAM outputs pixel mask

  For bin picking: Click on each part → instant segmentation
  No training needed!

DINO / DINOv2 (Meta, 2023):
────────────────────────────────────────
  • Self-supervised visual features
  • No labels needed for pre-training!
  • Features rival supervised ImageNet training
  • Excellent for transfer learning

MULTIMODAL MODELS (e.g., GPT-4V, Gemini):
────────────────────────────────────────
  • Process BOTH text AND images
  • "Find the bolts in this image and count them"
  • Natural language interface to vision!

  Potential: Describe a new part in words → system finds it
```

3.5 Comparing Approaches Across Eras

Evolution Summary

```
CV APPROACHES: COMPLETE COMPARISON
═══════════════════════════════════════════════════
                    ERA 1          ERA 2          ERA 3
                    CLASSICAL CV   FEATURE+ML     DEEP LEARNING
                    (1960-1995)    (1995-2012)    (2012-now)
──────────────────────────────────────────────────────────────
Knowledge       Hand-designed  Hand-crafted   Learned from
Source          algorithms     features       raw pixels

Representation  Edges, shapes, SIFT, HOG,     Learned feature
                contours       BoVW vectors   maps (CNN/ViT)
```

```
Feature Design  Manual         Manual         Automatic
                (edge filters) (descriptors)  (end-to-end)

Training Data   Not needed     Hundreds to    Thousands to
Required                       thousands      millions

Typical         Canny, Hough,  SIFT+RANSAC,   ResNet, YOLO,
Methods         template match HOG+SVM, BoVW  Mask R-CNN, ViT

ImageNet        N/A            ~25% error     ~2% error
Accuracy

Bin Picking     Structured     Semi-random    Random piles
Capability      bins only      bins           ✓

Pick Success    70-85%         85-95%         95-99%+
Rate

Cycle Time      2-5 sec        5-15 sec       0.5-3 sec

Explainability  HIGH           MEDIUM         LOW
                (trace edges)  (feature viz)  (black box)

Adaptability    LOW            MEDIUM         HIGH
                (re-tune)      (new features) (fine-tune)
────────────────────────────────────────────────────────────────────────
```

When to Use What

```
CHOOSING A COMPUTER VISION APPROACH
═══════════════════════════════════════════════════════════

USE CLASSICAL CV WHEN:
──────────────────────────────────────────────────────
 ✓ Patterns are simple and geometric (lines, circles)
 ✓ Lighting is controlled and consistent
 ✓ Speed is critical on embedded hardware
 ✓ No training data available
 ✓ Need sub-pixel precision for measurement tasks
 ✓ Must explain every decision (regulated industry)

 Examples: Dimension measurement, barcode reading,
           simple defect detection, alignment

USE FEATURE ENGINEERING + ML WHEN:
──────────────────────────────────────────────────────
 ✓ Moderate data available
 ✓ Need invariance to rotation/scale
 ✓ Object matching (is this the same part?)
 ✓ Computational resources limited
```

```
 ✓ Domain-specific features are well understood

 Examples: Product matching, texture classification,
           fingerprint matching

USE DEEP LEARNING WHEN:
──────────────────────────────────────────────────
 ✓ Need highest accuracy
 ✓ Task is complex (many object types, cluttered scenes)
 ✓ Sufficient training data (or transfer learning)
 ✓ GPU available for inference
 ✓ Robustness to real-world variation needed

 Examples: Autonomous driving, random bin picking,
           medical imaging, quality inspection at scale

HYBRID APPROACHES (common in production):
──────────────────────────────────────────────────
 Deep learning detects → Classical CV measures precisely
 CNN segments object → Geometric algorithm fits 3D model
 YOLO finds parts → Template matching verifies identity

 Best of both worlds!
```

Summary and Key Takeaways

```
WEEK 7 SUMMARY
═══════════════════════════════════════════════════════

1. COMPUTER VISION IS HARD
   ──────────────────────────────────────────────────
   • Viewpoint, lighting, occlusion, scale variation
   • Same object → very different pixel patterns
   • Requires bridging the "semantic gap"

2. THREE ERAS OF COMPUTER VISION
   ──────────────────────────────────────────────────
   Era 1: Classical (edges, shapes, templates)
     Hand-designed algorithms, limited to simple scenes

   Era 2: Feature engineering + ML (SIFT, HOG, BoVW + SVM)
     Hand-designed features, learned classifiers

   Era 3: Deep learning (CNNs, object detectors, ViTs)
     Everything learned from data

3. KEY BREAKTHROUGHS
   ──────────────────────────────────────────────────
   • Edge detection: Mathematical image analysis
   • SIFT: Invariant local features
```

```
    • Bag of Visual Words: Borrowed from NLP
    • AlexNet (2012): CNN revolution
    • ResNet: Very deep networks via skip connections
    • YOLO: Real-time object detection
    • Transfer learning: Pre-train once, fine-tune anywhere
    • Vision Transformers: Attention for images

4. BIN PICKING AS A LENS
   ──────────────────────────────────────────────────
    • Structured bins (classical) → Semi-random (features+ML)
      → Fully random piles (deep learning)
    • Deep learning + synthetic data = current state of art
    • Still an active research frontier!

5. PRACTICAL WISDOM
   ──────────────────────────────────────────────────
    • Match approach to constraints (data, compute, accuracy)
    • Transfer learning dramatically reduces data requirements
    • Classical CV still valuable for measurement + verification
    • Hybrid systems combine strengths of multiple eras
    • Synthetic data can bootstrap training
```

References

Historical and Foundational

- Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd ed.). [Comprehensive CV textbook, free online]
- Forsyth, D. & Ponce, J. (2011). Computer Vision: A Modern Approach (2nd ed.).

Key Papers

- Lowe, D. (2004). "Distinctive Image Features from Scale-Invariant Keypoints" [SIFT]
- Dalal, N. & Triggs, B. (2005). "Histograms of Oriented Gradients for Human Detection" [HOG]
- Krizhevsky et al. (2012). "ImageNet Classification with Deep Convolutional Neural Networks" [AlexNet]
- He et al. (2016). "Deep Residual Learning for Image Recognition" [ResNet]
- Redmon et al. (2016). "You Only Look Once: Unified, Real-Time Object Detection" [YOLO]
- Dosovitskiy et al. (2021). "An Image is Worth 16x16 Words" [ViT] Kirillov et al. (2023). "Segment Anything" [SAM]

Bin Picking Specific

- Zeng et al. (2018). "Robotic Pick-and-Place of Novel Objects" [Dex-Net grasp planning]
- Wang et al. (2019). "DenseFusion: 6D Object Pose Estimation" [RGB-D fusion]